Esercizio 1

Il metodo statico *calcolaRimborso* della classe *TravelUtils* viene utilizzato per calcolare il rimborso da erogare ad un dipendente per un viaggio di lavoro. Il metodo prende in input quattro parametri:

- *kmPercorsi*: un float non negativo indicante la distanza (in km) dalla sede dell'azienda alla destinazione in cui si è svolto il viaggio di lavoro.
- *durata*: un intero non negativo che rappresenta la durata (in giorni) del viaggio di lavoro. I viaggi non possono avere durata superiore a 14 giorni.
- *isFestivo*: un booleano che indica se il viaggio copre o meno giorni festivi.
- *tipoDipendente*: una stringa che può assumere come valori: "DIRIGENTE", "FUNZIONARIO", "OPERATORE".

Il rimborso viene calcolato moltiplicando la durata del viaggio in giorni per un coefficiente determinato in base alla distanza e alla tipologia di dipendente, come da tabella seguente.

	$0 < kmPercorsi \leq 100$	$100 < kmPercorsi \leq 500$	$kmPercorsi > 500$
DIRIGENTE	100	150	200
FUNZIONARIO	75	100	150
OPERATORE	50	75	100

Se il viaggio copre anche giorni festivi, inoltre, viene sommato al rimborso calcolato come specificato sopra un ulteriore somma di € 200.

Se uno dei parametri non è valido, il metodo solleva una *ReimbursementException*.

a. Indicare, per ciascuno dei parametri del metodo *calcolaRimborso*, un partizionamento in classi di equivalenza per testing black-box;
b. Scrivere tre test JUnit con strategia Black Box per il metodo *calcolaRimborso*, indicando per ciascuno di essi quali classi di equivalenza copre. Si richiede inoltre che un test corrisponda a scenari in cui almeno uno dei parametri è non valido, e che i restanti due corrispondano a scenari in cui tutti i parametri sono validi. Ove possibile, i test devono coprire classi di equivalenza distinte.

Quanti test sono necessari per testare il metodo con strategia R-WECT? Motivare la risposta, descrivendo brevemente tale strategia.

Esercizio 2

Il metodo statico *calcolaIndennizzo* della classe *ServiziFerroviari* viene utilizzato per calcolare l'indennizzo da pagare ad un viaggiatore in seguito al ritardo di un treno ad alta velocità. Il metodo accetta i seguenti argomenti:

- *int minutiRitardo*: numero di minuti di ritardo del convoglio su cui viaggiava il passeggero;
- *String fidelity*: stringa che indica il livello di fidelity del cliente. Può assumere valori in "BASE" (clienti occasionali), "FREQUENT" (clienti che hanno acquistato almeno 10 biglietti nell'ultimo anno), o "BUSINESS" (clienti che hanno acquistato almeno 50 biglietti nell'ultimo anno);

Se almeno uno dei parametri è non valido, il metodo solleva una IllegalArgumentException. In caso contrario, il metodo ritorna un numero float rappresentante l'indennizzo (in Euro) da corrispondere al cliente. Tale indennizzo è calcolato moltiplicando il numero di minuti di ritardo per un coefficiente di rimborso determinato in base alla fidelity del cliente e all'entità del ritardo, come da tabella seguente.

		minutiRitardo		
		$\boldsymbol{minutiRitardo < 30}$	$\boldsymbol{30 \leq minutiRitardo < 60}$	$\boldsymbol{minutiRitardo \geq 60}$
fidelity	**BASE**	0	0	1
	FREQUENT	0	1	2
	BUSINESS	1	2	3

Per esempio, un passeggero con fidelity BASE il cui treno accumula un ritardo di 45 minuti ha diritto a $45 \times 0 = €\ 0{,}00$ di indennizzo. Un passeggero dello stesso treno con fidelity BUSINESS, invece, ha diritto a $45 \times 2 = €\ 90{,}00$ di indennizzo.

c. Indicare, per ciascuno dei parametri del metodo *calcolaIndennizzo*, un partizionamento in classi di equivalenza per testing black-box;
d. Scrivere tre test JUnit con strategia Black Box per il metodo *calcolaIndennizzo*, indicando per ciascuno di essi quali classi di equivalenza copre. Si richiede inoltre che un test corrisponda a scenari in cui almeno uno dei parametri è non valido, e che i restanti due corrispondano a scenari in cui tutti i parametri sono validi. Ove possibile, i test devono coprire classi di equivalenza distinte.
e. Quanti test sono necessari per testare il metodo con strategia R-WECT? Motivare la risposta, descrivendo brevemente le diverse strategie.

Esercizio 3

Il metodo statico *calcolaImposta* della classe *TaxUtils* calcola le imposte dovute in un sistema a tassazione progressiva con due scaglioni. Il metodo prende in input quattro parametri:

- *imponibile*: un float non negativo indicante la base imponibile, in Euro.
- *soglia*: un float non negativo indicante la soglia di imponibile che determina il passaggio dal primo al secondo scaglione.
- *aliquota1*: un intero compreso tra 0 e 100, estremi esclusi. Indica l'aliquota di tassazione per il primo scaglione.
- *aliquota2*: un intero compreso tra 0 e 100, estremi esclusi. Indica l'aliquota di tassazione per il secondo scaglione.

Per esempio, se l'imponibile è 1000€, la soglia è fissata a 700€, e le aliquote sono rispettivamente al 10% e al 30%, l'imposta dovuta sarà calcolata come segue. Per la parte di imponibile rientrante nel primo scaglione, ovvero non superiore alla soglia, si applica l'aliquota del 10%, corrispondente a 70€. Per la parte di imponibile che rientra nel secondo scaglione, ovvero i restanti 300€, si applica la seconda aliquota al 30%, corrispondente a 100€ di imposta. In totale, le imposte nell'esempio ammontano a 70€ + 100€ = 170€.

Se uno dei parametri non è valido, il metodo solleva una *TaxException*.

f. Indicare, per ciascuno dei parametri del metodo *calcolaImposta*, un partizionamento in classi di equivalenza per testing black-box;
g. Scrivere tre test JUnit con strategia Black Box per il metodo *calcolaImposta*, indicando per ciascuno di essi quali classi di equivalenza copre. Si richiede inoltre che un test corrisponda a scenari in cui almeno uno dei parametri è non valido, e che i restanti due corrispondano a scenari in cui tutti i parametri sono validi. Ove possibile, i test devono coprire classi di equivalenza distinte.

Quanti test sono necessari per testare il metodo con strategia SECT? Motivare la risposta, descrivendo brevemente tale strategia.

Esercizio 4

Il metodo statico `calculatePrize` della classe `SwEngGames` viene utilizzato in una università di Seoul per calcolare il montepremi finale da destinare ai singoli studenti che superano tutte le dure prove predisposte nell'ambito di un gioco a premi a tema Ingegneria del Software. Il metodo accetta i seguenti argomenti:

- `int nEliminated`: numero (non negativo) di studenti eliminati durante le prove;
- `int nWinners`: numero (non negativo) di studenti che hanno superato tutte le prove (vincitori);
- `String gameEdition`: stringa non `null` che indica che tipo di edizione del gioco a premi si sta considerando. Il parametro può assumere i seguenti valori: "XMAS" (versione natalizia), "EASTER" (versione pasquale), "DEFAULT" (versione base). Qualsiasi altra stringa diversa da `null` deve essere interpretata come "DEFAULT".

Se almeno uno dei parametri è non valido, il metodo solleva una `IllegalArgumentException`. In caso contrario, il metodo ritorna un numero float rappresentante il montepremi (in Won, la valuta ufficiale della Corea del Sud) che sarà destinato a ciascuno studente vincitore. Tale budget è calcolato come descritto di seguito. Innanzitutto, si calcola il budget complessivo, utilizzando la formula:

$$\text{overallBudget} = (\text{nEliminated} \times \text{coeffPrize})$$

Il coefficiente coeffPrize viene determinato in base all'edizione del gioco a premi, come da tabella seguente:

coeffPrize		
XMAS	**EASTER**	**DEFAULT**
10.000.000	2.000.000	1.000.000

Infine, il budget complessivo viene diviso in parti uguali tra tutti gli studenti che hanno superato tutte le prove. Per esempio, una invocazione del metodo con 10 studenti eliminati durante le prove, 2 studenti che hanno superato le prove e l'edizione del gioco "DEFAULT", risulterà in un budget complessivo di 10 milioni di Won, da suddividere tra due vincitori, portando a un risultato di 5 milioni di Won per studente vincitore.

a. Indicare, per ciascuno dei parametri del metodo `calculatePrize`, un partizionamento in classi di equivalenza per testing black-box;
b. Scrivere quattro test JUnit con strategia Black Box per il metodo `calculatePrize`, indicando per ciascuno di essi quali classi di equivalenza copre. Si richiede inoltre che un test corrisponda a scenari in cui almeno uno dei parametri è non valido, e che i restanti tre corrispondano a scenari in cui tutti i parametri sono validi. Ove possibile, i test devono coprire classi di equivalenza distinte.

Quanti test sono necessari per testare il metodo con strategia R-WECT? Motivare la risposta, descrivendo brevemente le diverse strategie.

Esercizio 5

Il metodo `isCharInString` riportato di seguito viene utilizzato per verificare se un certo carattere è presente all'interno di una data stringa.

```
 1  public static boolean isCharInString(String str, char target) {
 2      if (str == null) {
 3          throw new IllegalArgumentException("Invalid String!");
 4      }
 5
 6      if (str.isEmpty()) {
 7          throw new IllegalArgumentException("Invalid String!");
 8      }
 9
10      int i = 0;
11      while (true) {
12          if(i >= str.length() || str.charAt(i) == target) {
13              break;
14          }
15          i++;
16      }
17
18      return i < str.length();
19  }
```

a. Rappresentare il CFG del metodo `isCharInString`.
b. Scrivere quattro test JUnit con strategia White-Box per il metodo `isCharInString`, indicando per ciascuno di essi quale cammino copre nel CFG. Si richiede che almeno un test copra uno scenario di errore (lancio di una `IllegalArgumentException`) e che, ove possibile, i test coprano cammini distinti.
c. Qual è la *Test Effectiveness Ratio* (TER), relativamente alla copertura di nodi del CFG, di ciascuno dei test sviluppati al punto (b)? Qual è, invece, la TER relativa all'intera suite di test prodotta al punto (b)? Motivare la risposta, descrivendo brevemente cosa si intende per TER e a cosa serve tale metrica.

Esercizio 6

Il metodo statico *calcolaPrezzo* della classe *ServiziMuseali* viene utilizzato per calcolare il costo di un biglietto di ingresso in una struttura museale. Il metodo accetta i seguenti parametri:

- *int age*: età della persona che vuole acquistare il biglietto;
- *String type*: stringa che indica il livello di fidelity del cliente. Può assumere valori in "BASE" (clienti regolari), "UNIVERSITY" (per studenti universitari), o "GUIDES" (per guide e accompagnatori);
- *boolean protected:* indica se il cliente appartiene ad una delle categorie protette a norma di legge;

Se almeno uno dei parametri è non valido, il metodo solleva una IllegalArgumentException. In caso contrario, il metodo ritorna un numero float rappresentante il prezzo (in Euro) del biglietto. Tale prezzo è calcolato in base alla tipologia di cliente e in base alla sua età, come da tabella seguente.

		age		
		$\boldsymbol{age < 18}$	$\boldsymbol{18 \leq age < 65}$	$\boldsymbol{age \geq 65}$
type	**BASE**	0	10	5
	UNIVERSITY	0	7	0
	GUIDES	0	5	0

Per le categorie protette, inoltre, il biglietto di ingresso è sempre gratuito.

a. Indicare, per ciascuno dei parametri del metodo *calcolaPrezzo*, un partizionamento in classi di equivalenza per testing black-box;
b. Scrivere tre test JUnit con strategia Black Box per il metodo *calcolaPrezzo*, indicando per ciascuno di essi quali classi di equivalenza copre. Si richiede inoltre che un test corrisponda a scenari in cui almeno uno dei parametri è non valido, e che i restanti due corrispondano a scenari in cui tutti i parametri sono validi. Ove possibile, i test devono coprire classi di equivalenza distinte.
c. Quanti test sono necessari per testare il metodo con strategia R-WECT? Motivare la risposta, descrivendo brevemente tale strategia.

Esercizio 7

Il metodo `calculateBudget` riportato di seguito viene utilizzato per calcolare il budget da destinare ai regali per le festività di studenti di Informatica.

```
1  float calculateBudget(int nGA, int nBA, boolean studiedSoftwareEng){
2      int budget = 0;
3
4      if(nGA < 0 || nBA < 0){
5          throw new IllegalArgumentException("Invalid num. of good/bad actions");
6      }
7
8      for (int i = 0; i < nGA; i++) {
9          if(studiedSoftwareEng){
10             budget += 15; // Each good action adds €15 to the budget
11         } else {
12             budget += 5;  // Each good action adds €5 to the budget
13         }
14     }
15
16     for (int i = 0; i < nBA; i++) {
17         budget -= 10; // Each bad action deducts €10 from the budget
18     }
19
20     if (budget < 0) {
21         budget = 0;
22     }
23     return budget;
24 }
```

a. Rappresentare il CFG del metodo `calculateBudget`.
b. Scrivere quattro test JUnit con strategia White-Box per il metodo `calculateBudget`, indicando per ciascuno di essi quale cammino copre nel CFG. Si richiede che almeno un test copra uno scenario di errore (lancio di una `IllegalArgumentException`) e che, ove possibile, i test coprano cammini distinti.
c. Qual è la *Test Effectiveness Ratio* (TER), relativamente alla copertura di nodi del CFG, della suite di test sviluppata al punto (b)? Motivare la risposta, descrivendo brevemente cosa si intende per TER e a cosa serve tale metrica.

Esercizio 8

Il metodo `calculateBudget` della classe `PresentsUtils` viene utilizzato per calcolare il budget massimo da destinare a regali per le festività per uno studente di Informatica. Il metodo accetta come argomenti i seguenti parametri:

- `int nGA`: numero (non negativo) di buone azioni compiute dallo studente durante l'anno solare;
- `int nBA`: numero (non negativo) di cattive azioni compiute dallo studente durante l'anno solare;
- `boolean studiedSoftwareEngineering`: valore booleano che indica se la persona ha studiato o meno per l'esame di Ingegneria del Software.

Se almeno uno dei parametri è non valido, il metodo solleva una `IllegalArgumentException`. In caso contrario, il metodo ritorna un numero intero rappresentante il budget massimo (in €) che sarà destinato all'acquisto dei regali per lo studente. Tale budget massimo è calcolato come descritto di seguito. Innanzitutto, si calcola il budget di base, utilizzando la formula:

$$\text{baseBudget} = (\text{nGA} \times \text{coeffGood}) - (\text{nBA} \times \text{coeffBad})$$

I due coefficienti coeffGood e coeffBad sono determinati, rispettivamente, in base al numero di buone e cattive azioni, come da tabella seguente:

coeffGood		**coeffBad**	
$0 \leq \text{nGA} \leq 100$	$100 < \text{nGA}$	$0 \leq \text{nBA} \leq 10$	$10 < \text{nBA}$
1	2	1	3

Infine, se lo studente ha studiato per l'insegnamento di Ingegneria del Software, il budget massimo è calcolato moltiplicando per due il budget di base. Se invece lo studente non ha studiato, il budget finale sarà uguale al budget di base. Si noti che, in alcuni casi, il budget massimo può essere negativo.

a. Indicare, per ciascuno dei parametri del metodo `calculateBudget`, un partizionamento in classi di equivalenza per testing black-box;
b. Scrivere quattro test JUnit con strategia Black Box per il metodo `calculateBudget`, indicando per ciascuno di essi quali classi di equivalenza copre. Si richiede inoltre che un test corrisponda a scenari in cui almeno uno dei parametri è non valido, e che i restanti tre corrispondano a scenari in cui tutti i parametri sono validi. Ove possibile, i test devono coprire classi di equivalenza distinte.
c. Quanti test sono necessari per testare il metodo con strategia SECT? Quanti con strategia R-WECT? Quanti con Pair-wise coverage? Motivare la risposta, descrivendo brevemente le diverse strategie.

Esercizio 9

In un videogioco di ruolo a turni, il metodo statico `dodgeProbability` della classe `MissileAttacksUtils` viene utilizzato per calcolare la probabilità che il personaggio controllato dal giocatore riesca a schivare un proiettile magico con determinate caratteristiche. Il metodo prende in input i seguenti parametri:

- `double speed`: indica la velocità del proiettile, e può assumere valori nell'intervallo]0,100[;
- `boolean isTargetTracking`: indica se il proiettile possiede la proprietà di guida autonoma verso il bersaglio;
- `int dex`: indica il livello della statistica "dexterity" del giocatore, e può assumere valori interi non negativi;

Se i parametri non sono validi, il metodo solleva una `IllegalArgumentException`. In caso contrario, ritorna la probabilità del giocatore di schivare l'attacco. Tale proprietà è calcolata in maniera pseudo-casuale all'interno di intervalli determinati come mostrato nella tabella seguente. Gli intervalli sono da considerarsi con estremi inclusi.

	$0 < \text{speed} \leq 30$		$30 < \text{speed} \leq 80$		$80 < \text{speed} < 100$	
	Tracking	***Non Tracking***	***Tracking***	***Non Tracking***	***Tracking***	***Non Tracking***
$\boldsymbol{dex < 50}$	0.2 – 0.5	0.5 – 0.8	0.3 – 0.4	0.4 – 0.5	0	0
$\boldsymbol{50 \leq dex < 80}$	0.5 – 0.8	1	0.4 – 0.6	0.5 – 0.7	0	0.2 – 0.5
$\boldsymbol{dex \geq 80}$	1	1	0.7 – 0.8	0.8 – 1	0.2 – 0.5	0.3 – 0.6

Per esempio, qualsiasi probabilità di schivata compresa tra 0.2 e 0.5 sarebbe valida per un proiettile con velocità di 99, con la proprietà TargetTracking attiva, lanciato contro un giocatore con livello di dexterity di 81. Allo stesso modo, un proiettile con velocità compresa tra 80 e 100 (estremi esclusi) non può mai essere schivato da un giocatore con dexterity inferiore a 50 (probabilità di schivata è fissa a 0).

a) Indicare, per ciascuno dei parametri del metodo `dodgeProbability`, le classi di equivalenza individuate.
b) Scrivere quattro test JUnit con strategia Black Box per il metodo `dodgeProbability`, indicando per ciascuno di essi quali classi di equivalenza copre. Si richiede inoltre che un test corrisponda a scenari in cui i parametri non sono validi, e che i restanti tre corrispondano a scenari in cui i parametri sono validi.
c) Quanti test sono necessari per testare il metodo con strategia N-WECT? Motivare la risposta.

Esercizio 10

Un numero è palindromico se può essere letto allo stesso modo sia da sinistra a destra che da destra a sinistra. Ad esempio, 121, 12321 e 4884 sono numeri palindromici. Il metodo isPalindrome riportato di seguito prende come argomento un intero non negativo, e ritorna vero se tale intero è palindromico e falso altrimenti.

```
1   public static boolean isPalindrome(int num) {
2       if (num < 0) {
3           throw new BarException("Negative numbers are not accepted!");
4       }
5       int original = num;
6       int reverse = 0;
7       while (true) {
8           if(num==0){
9               break;
10          }
11          int digit = num % 10;
12          reverse = reverse * 10 + digit;
13          num /= 10;
14      }
15      return original == reverse;
16  }
```

a) Rappresentare il CFG del metodo isPalindrome;
b) Scrivere tre test JUnit con strategia White Box per il metodo isPalindrome, indicando per ciascuno di essi quale cammino copre nel CFG. Si richiede che almeno un test copra uno scenario di errore (input non valido) e che, ove possibile, i test JUnit coprano cammini distinti nel CFG;
c) Qual è la *Test Effectiveness Ratio* (TER), relativamente alla copertura di nodi del CFG, della suite di test sviluppata al punto (b)? Motivare la risposta, descrivendo brevemente cosa si intende per TER e a cosa serve tale metrica.